

Batch Normalization and Programming framework

Clarification about Upcoming Normalizing Activations in a Network Video

Please note, as pointed out earlier in Week 1, the normalization **should** be: $X = X/\sigma$

Normalizing Activations in a Network

Batch normalization makes your hyperparameter search problem much easier, makes your neural network much more robust.

The choice of hyperparameters is a much bigger range of hyperparameters that work well, and will also enable you to much more easily train even very deep networks.

Batch norm make your model stable

Implementing Batch Norm

Given some intermediate values in NN

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$
$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$
$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
$$\hat{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

Use $\hat{z}^{(i)}$ instl of $z^{(i)}$.

If $\gamma = \sqrt{\sigma^2 + \epsilon}$
 $\beta = \mu$
then $\hat{z}^{(i)} = z^{(i)}$

learnable parameters of model.

Andrew Ng

So the intuition I hope you'll take away from this is that we saw how normalizing the input features x can help learning in a neural network.

And what batch norm does is it applies that normalization process not just to the input layer, but to the values even deep in some hidden layer in the neural network. So it will apply this type of normalization to normalize the mean and variance of some of your hidden units' values, z . But one difference between the training input and these hidden unit values is you might not want your hidden unit values be forced to have mean 0 and variance 1. For example, if you have a sigmoid activation function, you don't want your values to always be clustered here.

You might want them to have a larger variance or have a mean that's different than 0, in order to better take advantage of the nonlinearity of the sigmoid function rather than have all your values be in just this linear regime. So that's why with the parameters γ and β , you can now make sure that your z_i values have the range of values that you want. But what it does really is it then shows that your hidden units have standardized mean and variance, where the mean and variance are controlled by two explicit parameters γ and β which the learning algorithm can set to whatever it wants.

Implementing Batch Norm

Given some intermediate values in NN $z^{(1)}, \dots, z^{(m)}$

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\hat{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

Use $\hat{z}^{(i)}$ instl of $z^{(i)}$.

If $\gamma = \sqrt{\sigma^2 + \epsilon}$
 $\beta = \mu$
 then $\hat{z}^{(i)} = z^{(i)}$

learnable parameters of model.

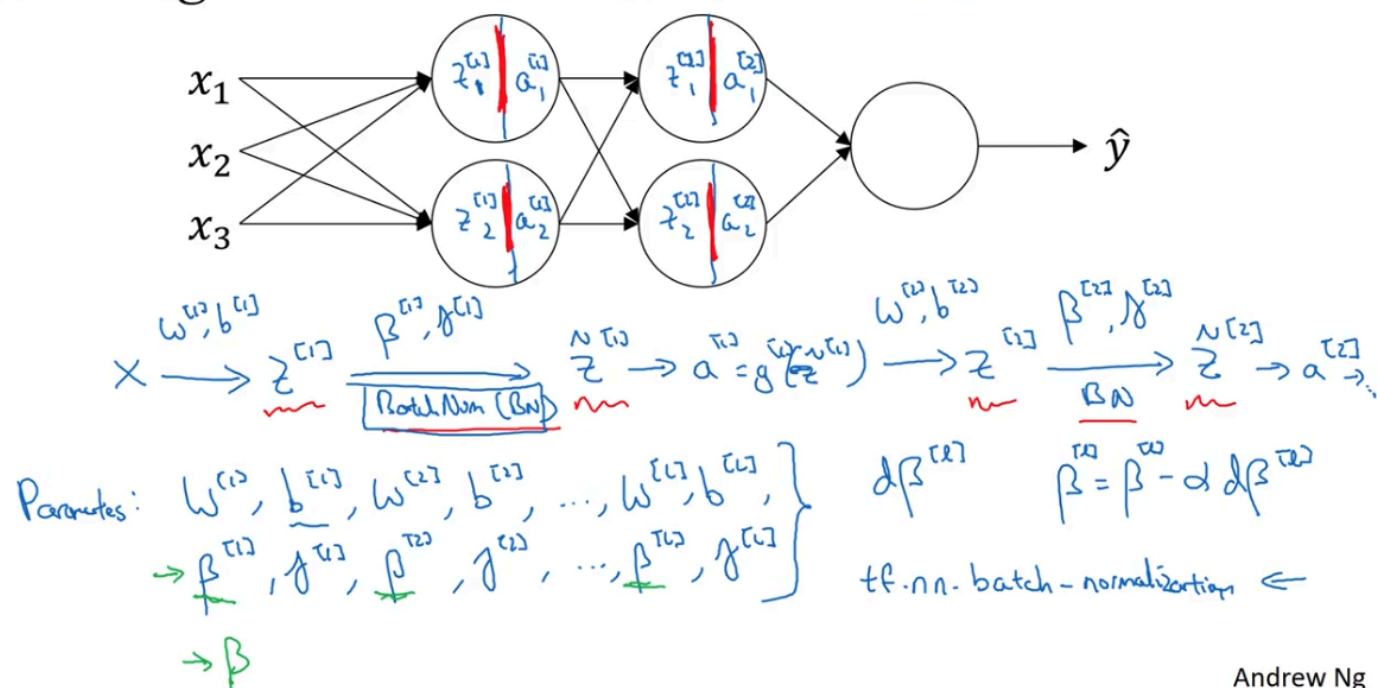
$X \leftarrow$
 $z^{(i)} \leftarrow$

Andrew Ng

Batch norm can be controlled by γ, β

Fitting Batch Norm into a Neural Network

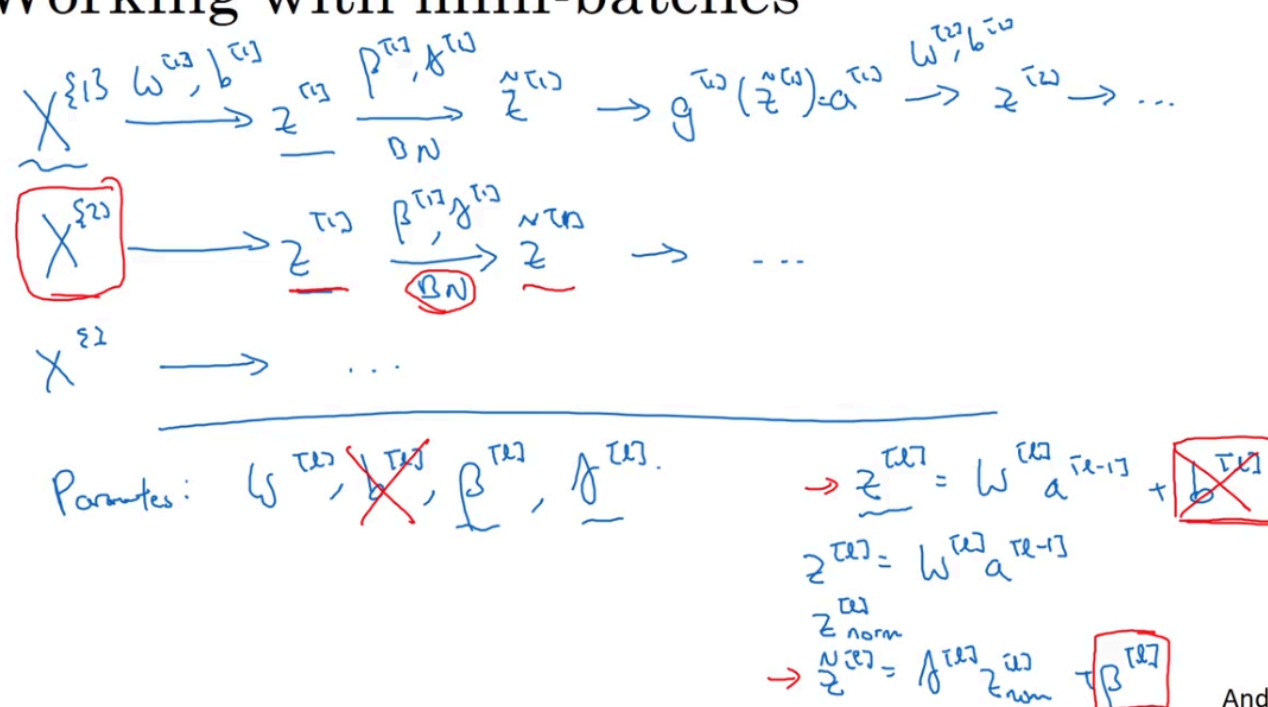
Adding Batch Norm to a network



Andrew Ng

Green beta is for momentum hyperparameter

Working with mini-batches



we can omit b in when using batch norm because after find the mean and let $x - \mu$, the b is subtracted by itself $\rightarrow$ no need to use b parameter

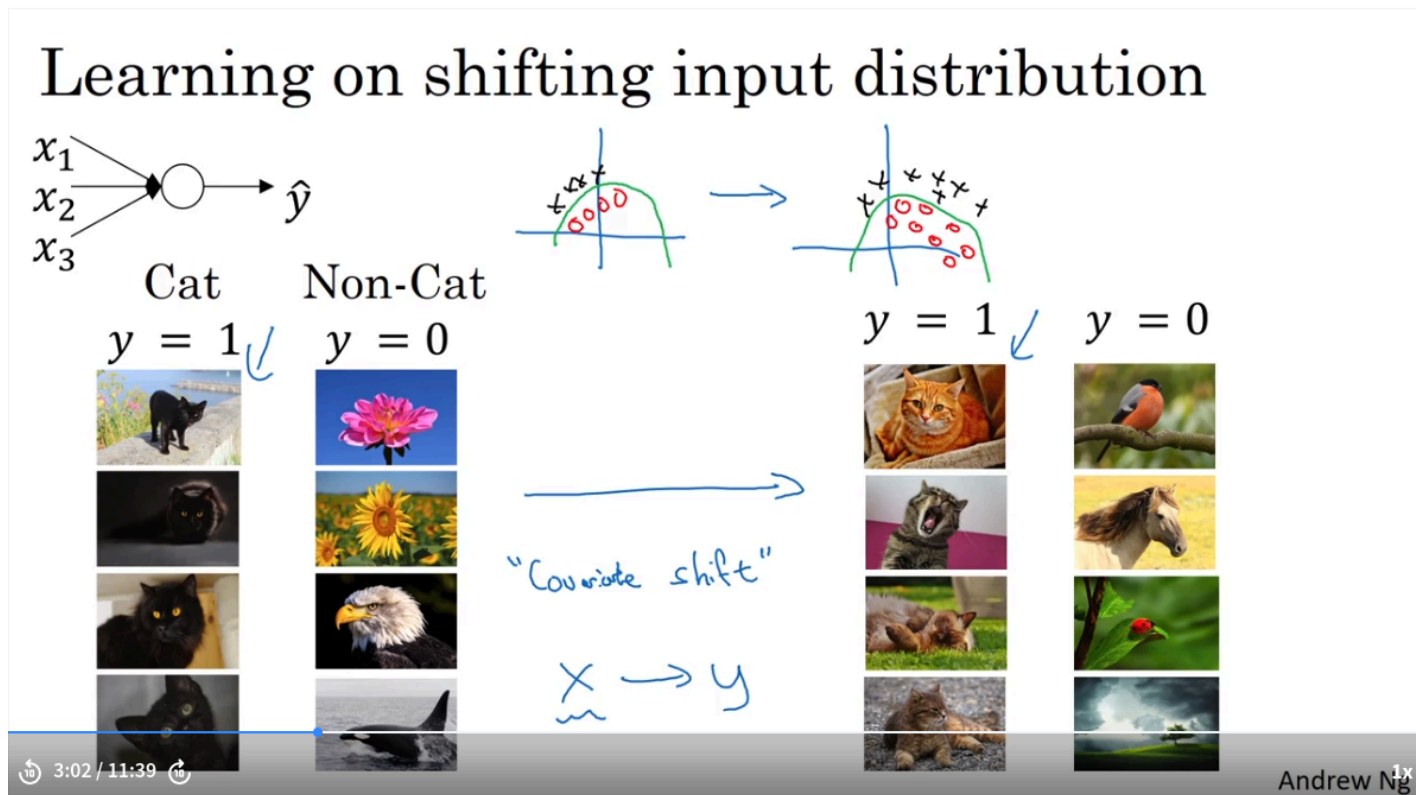
Implementing gradient descent with batch norm

Implementing gradient descent

for $t = 1 \dots \text{num Mini Batches}$
 Compute forward pass on $X^{[t]}$.
 In each hidden layer, use BN to replace $z^{[t]}$ with $z^{[t]}_{norm}$.
 Use backprop & compute $\frac{dL}{dW^{[t]}}, \frac{dL}{d\beta^{[t]}}, \frac{dL}{d\gamma^{[t]}}$
 Update params $\left. \begin{aligned} W^{[t]} &:= W^{[t]} - \alpha \frac{dL}{dW^{[t]}} \\ \beta^{[t]} &:= \beta^{[t]} - \alpha \frac{dL}{d\beta^{[t]}} \\ \gamma^{[t]} &:= \gamma^{[t]} - \alpha \frac{dL}{d\gamma^{[t]}} \end{aligned} \right\}$
 Works w/ momentum, RMSprop, Adam.

Andrew Ng

Why does Batch Norm work?



the left Oxy axis is for the model trained with black cat

the right Oxy axis is for the model trained with colored cat

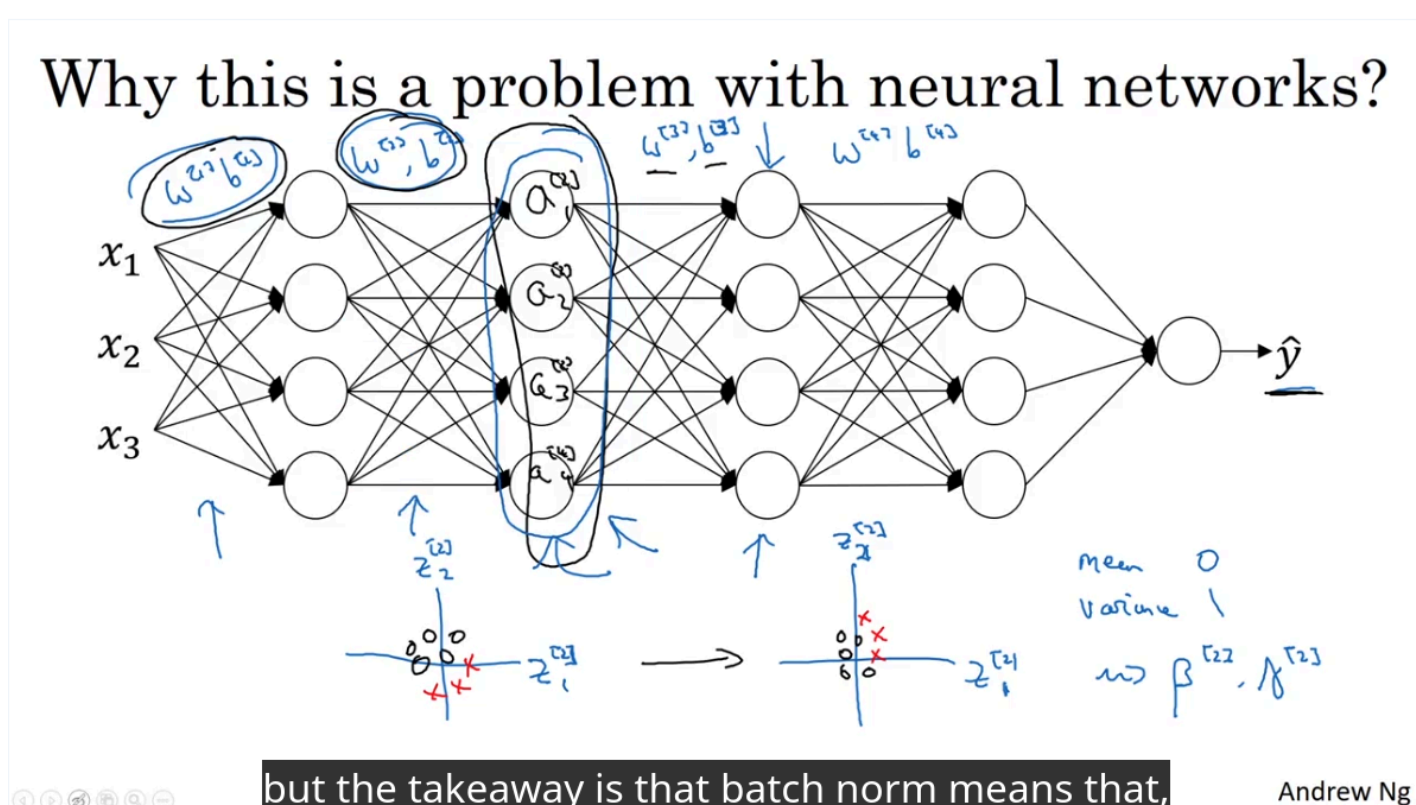
We cannot want the model trained with black cats perform well on the dataset with color cats

Even though there might be the same function that actually works well, but you wouldn't expect your learning algorithm to discover that green decision boundary, just looking at the data on the left.

So, this idea of your data distribution changing goes by the somewhat fancy name, covariate shift. And the idea is that, if you've learned some X to Y mapping, **if the distribution of X changes (the black cats to the color cats)**, then you might need to retrain your learning algorithm.

And this is true even if the function, the ground true function, mapping from X to Y , remains unchanged, which it is in this example, because the ground true function is this picture a cat or not. And the need to retrain your function becomes even more accurate or it becomes even worse if the ground true function shifts as well.

How does this problem of covariate shift apply to a neural network?



It limits the amount to which updating the parameters in the earlier layers can affect the distribution of values that the third layer now sees and therefore has to learn on.

And so, batch norm reduces the problem of the input values changing, it really causes these values to become more stable, so that the later layers of the neural network has more firm ground to stand on.

Even though the input distribution changes a bit, it changes less, and what this does is, even as the earlier layers keep learning, the amounts that this forces the later layers to adapt to as early as layer changes is reduced or, if you will, it weakens the coupling between what the early layers parameters has to do and what the later layers parameters have to do. And so it allows each layer of the network to learn by itself, a little bit more independently of other layers, and this has the effect of speeding up of learning in the whole network.

So I hope this gives some better intuition, but the takeaway is that batch norm means that, especially from the perspective of one of the later layers of the neural network, the earlier layers don't get to shift around as much, because they're constrained to have the same mean and variance.

Batch Norm as regularization

- Each mini-batch is scaled by the mean/variance computed on just that mini-batch.
- This adds some noise to the values $z^{[l]}$ within that minibatch. So similar to dropout, it adds some noise to each hidden layer's activations.
- This has a slight regularization effect.

Because the noise added is quite small,

Andrew Ng

So similar to dropout, it adds some noise to each hidden layer's activations. The way dropout has noises, it takes a hidden unit and it multiplies it by zero with some probability.

And multiplies it by one with some probability. And so your dropout has multiple of noise because it's multiplied by zero or one, whereas batch norm has multiples of noise because of scaling by the standard deviation, as well as additive noise because it's subtracting the mean. Well, here the estimates of the mean and the standard deviation are noisy.

And so, similar to dropout, batch norm therefore has a slight regularization effect. Because by adding noise to the hidden units, it's forcing the downstream hidden units not to rely too much on any one hidden unit. And so similar to dropout, it adds noise to the hidden layers and therefore has a very slight regularization effect. Because the noise added is quite small, this is not a huge regularization effect, and you might choose to use batch norm together with dropout, and you might use batch norm together with dropouts if you want the more powerful regularization effect of dropout.

And maybe one other slightly non-intuitive effect is that, if you use a bigger mini-batch size, right, so if you use use a mini-batch size of, say, 512 instead of 64, by using a larger mini-batch size, you're reducing this noise and therefore also reducing this regularization effect.

So that's one strange property of dropout which is that by using a bigger mini-batch size, you reduce the regularization effect. Having said this, I wouldn't really use batch norm as a regularizer, that's really not the intent of batch norm, but sometimes it has this extra intended or unintended effect on your learning algorithm. But, really, **don't turn to batch norm as a regularization**. Use it as a way to normalize your hidden units activations and therefore speed up learning. And I think the regularization is an almost unintended side effect.

Batch Norm at Test Time

Batch Norm at test time

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$

$$z_{\text{norm}}^{(i)} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\tilde{z}^{(i)} = \gamma z_{\text{norm}}^{(i)} + \beta$$

μ, σ^2 : estimate using exponentially weighted average (across mini-batches).

$x^{(1)}, x^{(2)}, x^{(3)}, \dots$

$\downarrow$

$\mu^{(1)}, \mu^{(2)}, \mu^{(3)}, \dots \rightarrow \mu$

$\sigma^{(1)}, \sigma^{(2)}, \sigma^{(3)}, \dots \rightarrow \sigma^2$

$\tilde{z}_{\text{norm}} = \frac{z - \mu}{\sqrt{\sigma^2 + \epsilon}}$

$\tilde{z} = \gamma \tilde{z}_{\text{norm}} + \beta$

Andrew Ng

you could in theory run your whole training set through your final network to get mu and sigma squared.

But in practice, what people usually do is implement an exponentially weighted average where you just keep track of the mu and sigma squared values you're seeing during training and use an exponentially weighted average, also sometimes called the running average, to just get a rough estimate of mu and sigma squared and then you use those values of mu and sigma squared at test time to do the scale and you need the gamma and beta values Z.

Deep Learning Frameworks

Deep learning frameworks

- Caffe/Caffe2
- CNTK
- DL4J
- Keras
- Lasagne
- mxnet
- PaddlePaddle
- TensorFlow
- Theano
- Torch

Choosing deep learning frameworks

- Ease of programming (development and deployment)
- Running speed
- Truly open (open source with good governance)

and that means both developing the neural network and

Andrew Ng

Tensorflow

```

In [1]: import numpy as np
import tensorflow as tf

In [2]: w = tf.Variable(0, dtype=tf.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def train_step():
    with tf.GradientTape() as tape:
        cost = w ** 2 - 10 * w + 25
        trainable_variables = [w]
        grads = tape.gradient(cost, trainable_variables)
        optimizer.apply_gradients(zip(grads, trainable_variables))

    print(w)

<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=0.0>

In [3]: train_step()
print(w)

<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=0.09999997>

In [4]: for i in range(1000):
        train_step()
        print(w)

<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=5.000001>

In [4]: for i in range(1000):
        train_step()
        print(w)

<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=5.000001>

In [7]: w = tf.Variable(0, dtype=tf.float32)
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def training(x, w, optimizer):
    def cost_fn():
        return x[0] * w ** 2 + x[1] * w + x[2]
    for i in range(1000):
        optimizer.minimize(cost_fn, [w])

    return w

w = training(x, w, optimizer)
print(w)

<tf.Variable 'Variable:0' shape=() dtype=float32, numpy=5.000001>

In [ ]:

```

There you go, and now W is

Code example

```

import numpy as np
import tensorflow as tf

w = tf.Variable(0, dtype=tf.float32)
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def training(x, w, optimizer):
    def cost_fn():
        return x[0] * w ** 2 + x[1] * w + x[2]
    for i in range(1000):
        optimizer.minimize(cost_fn, [w])
    return w

w = training(x, w, optimizer)

```

Code example



```
import numpy as np
import tensorflow as tf

w = tf.Variable(0, dtype=tf.float32)
x = np.array([1.0, -10.0, 25.0], dtype=np.float32)
optimizer = tf.keras.optimizers.Adam(0.1)

def training(x, w, optimizer):
    def cost_fn():
        return x[0] * w ** 2 + x[1] * w + x[2]
    for i in range(1000):
        optimizer.minimize(cost_fn, [w])
    return w

w = training(x, w, optimizer)
```

it uses computational graph

To learn more about Gradient Tape, you can take

[Course 2: Custom and Distributed Training with TensorFlow](#)

of our

[TensorFlow: Advanced Techniques Specialization](#)